

MODULE 3:

Azure Networking

Basics of Virtual Networks – Address Spaces, Subnets, DNS Servers – Creating and Using Virtual Networks – Network Security Groups – Virtual Appliances – Load Balancer basics – Configuring Load Balancers – Creating and Using Load balancers – Azure VPN - Windows Azure Network Services - Virtual Network- Traffic Manager - Using Traffic Manager.

Day 1: Virtual Network Fundamentals (60 minutes)

Task 1: Understanding Address Spaces and CIDR Notation

Research and Document:

1. Explain CIDR notation (e.g., 10.0.0.0/16, 192.168.1.0/24)
2. Calculate the following:
 - How many IP addresses in 10.0.0.0/16?
 - How many IP addresses in 10.1.0.0/24?
 - What's the range of 172.16.0.0/20?
3. Why does Azure reserve 5 IP addresses in each subnet?

Task 2: Create Your First Virtual Network

VNet 1: Production Network

- Resource Group: Networking-Lab-RG
- Name: VNet-Production
- Region: Your nearest region
- Address space: 10.0.0.0/16

Create the following subnets:

Subnet Name	Address Range	Purpose
WebTier-Subnet	10.0.1.0/24	Web servers
AppTier-Subnet	10.0.2.0/24	Application servers
DataTier-Subnet	10.0.3.0/24	Database servers
Management-Subnet	10.0.10.0/24	Management/Jump servers

Document:

- Screenshot of the virtual network overview
- Screenshot showing all subnets with their address ranges
- Number of available IPs in each subnet

Task 3: Configure DNS Settings

1. In VNet-Production, configure custom DNS servers:
 - Primary DNS: 10.0.10.10
 - Secondary DNS: 10.0.10.11
2. **Questions:**
 - What is the default DNS server in Azure VNets?
 - When would you use custom DNS servers?
 - What is Azure-provided DNS?

Task 4: Create a Second Virtual Network

VNet 2: Development Network

- Name: VNet-Development
- Region: Same as VNet-Production
- Address space: 10.1.0.0/16

Create subnets:

- Dev-Subnet: 10.1.1.0/24
 - Test-Subnet: 10.1.2.0/24
-

Day 2: Deploying Resources in Virtual Networks (70 minutes)

Task 1: Deploy Virtual Machines in Different Subnets

Create the following VMs in VNet-Production:

VM1: Web Server

- Name: WebServer-VM01
- Subnet: WebTier-Subnet
- OS: Windows Server 2022 or Ubuntu 20.04
- Size: Standard_B2s
- Public IP: Yes (for testing)
- Install IIS/Apache during deployment

VM2: App Server

- Name: AppServer-VM01
- Subnet: AppTier-Subnet
- OS: Same as WebServer
- Size: Standard_B2s
- Public IP: No
- Private IP: 10.0.2.10 (Static)

VM3: Database Server

- Name: DBServer-VM01
- Subnet: DataTier-Subnet
- OS: Windows Server 2022
- Size: Standard_B2s
- Public IP: No
- Private IP: 10.0.3.10 (Static)

VM4: Jump/Bastion Server

- Name: JumpBox-VM01
- Subnet: Management-Subnet
- OS: Windows Server 2022
- Size: Standard_B2s
- Public IP: Yes

Task 2: Test Internal Connectivity

1. RDP/SSH to JumpBox-VM01

2. From JumpBox, test connectivity to:
 - WebServer-VM01 (ping by private IP)
 - AppServer-VM01 (ping 10.0.2.10)
 - DBServer-VM01 (ping 10.0.3.10)
3. Document ping results and response times

Task 3: Verify Network Configuration

For each VM, document:

- Private IP address
- Public IP address (if applicable)
- DNS servers configured
- Default gateway
- Effective routes

Screenshot Required: Network interface configuration for each VM

Day 3: Network Security Groups (NSGs) (80 minutes)

Task 1: Create and Configure NSGs

Create separate NSGs for each tier:

NSG 1: WebTier-NSG

Inbound Security Rules:

Priority	Name	Port	Protocol	Source	Destination	Action
100	Allow-HTTP	80	TCP	Internet	VirtualNetwork	Allow
110	Allow-HTTPS	443	TCP	Internet	VirtualNetwork	Allow
120	Allow-RDP-From-Mgmt	3389	TCP	10.0.10.0/24	VirtualNetwork	Allow
4096	Deny-All-Inbound	*	*	*	*	Deny

NSG 2: AppTier-NSG

Inbound Security Rules:

Priority	Name	Port	Protocol	Source	Destination	Action
100	Allow-From-WebTier	8080	TCP	10.0.1.0/24	VirtualNetwork	Allow
110	Allow-RDP-From-Mgmt	3389	TCP	10.0.10.0/24	VirtualNetwork	Allow
4096	Deny-All-Inbound	*	*	*	*	Deny

NSG 3: DataTier-NSG

Inbound Security Rules:

Priority	Name	Port	Protocol	Source	Destination	Action
100	Allow-SQL-From-AppTier	1433	TCP	10.0.2.0/24	VirtualNetwork	Allow
110	Allow-RDP-From-Mgmt	3389	TCP	10.0.10.0/24	VirtualNetwork	Allow
4096	Deny-All-Inbound	*	*	*	*	Deny

NSG 4: Management-NSG

Inbound Security Rules:

Priority	Name	Port	Protocol	Source	Destination	Action
100	Allow-RDP-YourIP	3389	TCP	[Your IP]/32	VirtualNetwork	Allow
4096	Deny-All-Inbound	*	*	*	*	Deny

Task 2: Associate NSGs

Associate each NSG to its corresponding subnet

Task 3: Test NSG Rules

1. **Test Web Tier Access:**
 - Try to access WebServer-VM01 via HTTP from internet (should work)

- Try to RDP directly to WebServer-VM01 from internet (should fail)
- Try to RDP from JumpBox to WebServer (should work)
- 2. **Test App Tier Isolation:**
 - Try to access AppServer-VM01 from internet (should fail)
 - From WebServer, try to connect to AppServer on port 8080
 - From JumpBox, RDP to AppServer (should work)
- 3. **Test Database Tier Security:**
 - Try to access DBServer from internet (should fail)
 - From AppServer, test SQL port 1433 to DBServer (use telnet or Test-NetConnection)
 - From WebServer, try to access DBServer on port 1433 (should fail)

Document: Results of each test with screenshots

Task 4: NSG Flow Logs (Documentation)

Research and document:

1. What are NSG Flow Logs?
2. How to enable them?
3. What information do they capture?
4. Use cases for security auditing

Task 5: Application Security Groups (ASGs)

1. Create two Application Security Groups:
 - ASG-WebServers
 - ASG-AppServers
2. Assign VMs to ASGs:
 - WebServer-VM01 → ASG-WebServers
 - AppServer-VM01 → ASG-AppServers
3. Modify AppTier-NSG to use ASGs:
 - Change rule to allow traffic from ASG-WebServers to ASG-AppServers

Question: What are the benefits of ASGs over IP-based rules?

Day 4: VNet Peering and Connectivity (50 minutes)

Task 1: Configure VNet Peering

1. Create peering from VNet-Production to VNet-Development:
 - Name: Prod-to-Dev

- Allow virtual network access: Yes
 - Allow forwarded traffic: Yes
 - Allow gateway transit: No
2. Create peering from VNet-Development to VNet-Production:
 - Name: Dev-to-Prod
 - Allow virtual network access: Yes
 - Allow forwarded traffic: Yes
 - Use remote gateways: No

Screenshot Required: Both peering configurations showing status as "Connected"

Task 2: Deploy VM in Development VNet

1. Create a VM in VNet-Development:
 - Name: DevTest-VM01
 - Subnet: Dev-Subnet
 - Private IP: 10.1.1.10
 - Public IP: Yes (for testing)

Task 3: Test Cross-VNet Connectivity

1. From DevTest-VM01, ping:
 - WebServer-VM01 (10.0.1.x)
 - AppServer-VM01 (10.0.2.10)
 - DBServer-VM01 (10.0.3.10)
2. Document connectivity results

Questions:

- What is the latency between VMs in peered VNets?
 - Can you peer VNets in different regions?
 - What are the costs associated with VNet peering?
 - What is transitive peering and why doesn't Azure support it by default?
-

Day 5: Load Balancer Implementation (90 minutes)

Task 1: Create Backend Web Servers

Create two additional web servers:

VM5: WebServer-VM02

- Subnet: WebTier-Subnet

- Private IP: 10.0.1.11 (Static)
- Public IP: No
- Install IIS/Apache with custom homepage: "Server 1"

VM6: WebServer-VM03

- Subnet: WebTier-Subnet
- Private IP: 10.0.1.12 (Static)
- Public IP: No
- Install IIS/Apache with custom homepage: "Server 2"

Task 2: Create Azure Load Balancer

Load Balancer Configuration:

- Name: WebTier-LB
- Type: Public
- SKU: Standard
- Tier: Regional

Frontend IP Configuration:

- Name: LB-Frontend-IP
- IP type: Public IP address
- Create new public IP: LB-PublicIP (Static)

Backend Pool:

- Name: WebServers-BackendPool
- Virtual network: VNet-Production
- Add VMs:
 - WebServer-VM01
 - WebServer-VM02
 - WebServer-VM03

Task 3: Configure Health Probes

Create health probe:

- Name: HTTP-HealthProbe
- Protocol: HTTP
- Port: 80
- Path: /
- Interval: 5 seconds
- Unhealthy threshold: 2 consecutive failures

Task 4: Create Load Balancing Rules

Rule 1: HTTP Load Balancing

- Name: HTTP-LB-Rule
- Frontend IP: LB-Frontend-IP
- Protocol: TCP
- Port: 80
- Backend port: 80
- Backend pool: WebServers-BackendPool
- Health probe: HTTP-HealthProbe
- Session persistence: None
- Idle timeout: 4 minutes
- Floating IP: Disabled

Rule 2: HTTPS Load Balancing (if implementing SSL)

- Name: HTTPS-LB-Rule
- Port: 443
- Backend port: 443
- (Other settings same as HTTP rule)

Task 5: Test Load Balancer

1. Access the Load Balancer's public IP via browser
2. Refresh multiple times and observe:
 - Which server responds (check custom homepage)
 - Distribution pattern
3. Document the load distribution

Testing Scenarios:

1. **Normal Operation:**
 - Access LB IP 20 times
 - Count responses from each server
2. **Server Failure Simulation:**
 - Stop WebServer-VM02
 - Continue accessing LB IP
 - Verify traffic only goes to VM01 and VM03
 - Start VM02 and verify it rejoins the pool
3. **Session Persistence:**
 - Change session persistence to "Client IP"
 - Test from same browser
 - Verify you always hit the same backend server

Screenshots Required:

- Load balancer overview
- Backend pool health status
- Browser showing responses from different servers

Task 6: Configure Inbound NAT Rules

Create NAT rules for management access:

NAT Rule Name	Frontend Port	Backend Port	Target VM
RDP-to-Web 01	50001	3389	WebServer-V M01
RDP-to-Web 02	50002	3389	WebServer-V M02
RDP-to-Web 03	50003	3389	WebServer-V M03

Test accessing each server via LB-PublicIP:50001, 50002, 50003

Day 6: Internal Load Balancer (40 minutes)

Task 1: Create Internal Load Balancer

ILB Configuration:

- Name: AppTier-ILB
- Type: Internal
- SKU: Standard
- Virtual network: VNet-Production
- Subnet: AppTier-Subnet
- Frontend private IP: 10.0.2.100 (Static)

Task 2: Configure Backend Pool and Rules

1. Backend Pool: Add AppServer-VM01 (and additional app servers if created)
2. Health Probe: TCP probe on port 8080
3. Load Balancing Rule: Port 8080 to backend port 8080

Task 3: Test Internal Load Balancer

1. From WebServer-VM01, connect to internal LB IP (10.0.2.100:8080)
2. Verify connectivity
3. Document the traffic flow

Questions:

- When would you use an Internal Load Balancer?
 - Can VMs in different VNets access an ILB? (Test with DevTest-VM01)
 - What's the difference between Basic and Standard SKU load balancers?
-

Day 7: Azure VPN Gateway (70 minutes)

Task 1: Create VPN Gateway Subnet

In VNet-Production, create:

- Subnet name: GatewaySubnet (must be exact name)
- Address range: 10.0.255.0/27

Task 2: Create Virtual Network Gateway

VPN Gateway Configuration:

- Name: VNet-Production-VPNGateway
- Gateway type: VPN
- VPN type: Route-based
- SKU: VpnGw1
- Virtual network: VNet-Production
- Public IP address: Create new (VPNGateway-PIP)
- Enable active-active mode: No
- Configure BGP: No

Note: Gateway creation takes 30-45 minutes. While waiting, proceed to documentation tasks below.

Task 3: Site-to-Site VPN Documentation (While Gateway Creates)

Research and document:

1. **VPN Gateway SKUs:**
 - Compare Basic, VpnGw1, VpnGw2, VpnGw3
 - Throughput limits
 - Tunnel count limits
 - Pricing differences
2. **VPN Connectivity Options:**
 - Site-to-Site (S2S)
 - Point-to-Site (P2S)
 - VNet-to-VNet
 - ExpressRoute
3. **Create a diagram showing:**
 - On-premises network (simulated)
 - VPN Gateway
 - Azure VNet
 - Traffic flow

Task 4: Configure Point-to-Site VPN

Once gateway is created:

1. **Configure P2S settings:**
 - Address pool: 172.16.0.0/24
 - Tunnel type: OpenVPN (SSL) or IKEv2
 - Authentication type: Azure certificate
2. **Generate Certificates:** For Windows (PowerShell):

powershell

5. *# Generate root certificate*
6. `$cert = New-SelfSignedCertificate -Type Custom -KeySpec Signature ``
7. `-Subject "CN=P2SRootCert" -KeyExportPolicy Exportable ``
8. `-HashAlgorithm sha256 -KeyLength 2048 ``
9. `-CertStoreLocation "Cert:\CurrentUser\My" -KeyUsageProperty Sign -KeyUsage`
`CertSign`
- 10.
11. *# Generate client certificate*
12. `New-SelfSignedCertificate -Type Custom -DnsName P2SChildCert -KeySpec`
`Signature ``
13. `-Subject "CN=P2SChildCert" -KeyExportPolicy Exportable ``
14. `-HashAlgorithm sha256 -KeyLength 2048 ``
15. `-CertStoreLocation "Cert:\CurrentUser\My" ``

-Signer \$cert -TextExtension @("2.5.29.37={text}1.3.6.1.5.5.7.3.2")

3. **Upload root certificate** public key to Azure
4. **Download VPN client** and install
5. **Test Connection:**
 - Connect via VPN client
 - Get IP from 172.16.0.0/24 range
 - Ping VMs in VNet-Production
 - Access internal resources

Screenshots Required:

- VPN Gateway overview
- P2S configuration
- Connected VPN client showing assigned IP
- Successful ping to internal VM

Task 5: VNet-to-VNet VPN (Optional)

If time and budget allow:

1. Create a VPN Gateway in VNet-Development
 2. Configure VNet-to-VNet connection
 3. Test connectivity
 4. Compare with VNet Peering (speed, latency, cost)
-

Task 6: Azure Firewall (Documentation)

Research and document:

1. **Azure Firewall features:**
 - FQDN filtering
 - Network rules
 - Application rules
 - Threat intelligence
 - DNS proxy
2. **Compare Azure Firewall vs NVA:**
 - Capabilities
 - Management overhead
 - Costs
 - Use cases for each

3. **Design a hub-spoke topology:**

- Hub VNet with Azure Firewall
- Multiple spoke VNets
- UDRs forcing traffic through firewall
- Create architecture diagram

Task 7: DDoS Protection

Research and document:

1. **DDoS Protection tiers:**

- Basic (free)
- Standard (paid)
- Feature comparison

2. **Attack types protected against:**

- Volumetric attacks
- Protocol attacks
- Application layer attacks

3. **Cost-benefit analysis:**

- When to enable Standard tier?
 - Cost vs potential loss from attack
-